

MINI PROJECT

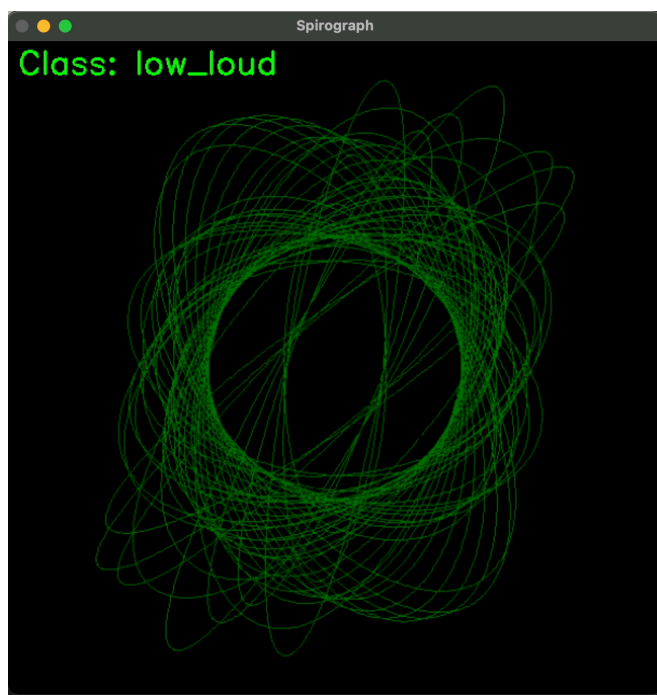
AUDIO-REACTIVE

SPIROGRAPHS AND THEIR

SOUNDS

PIDDIU

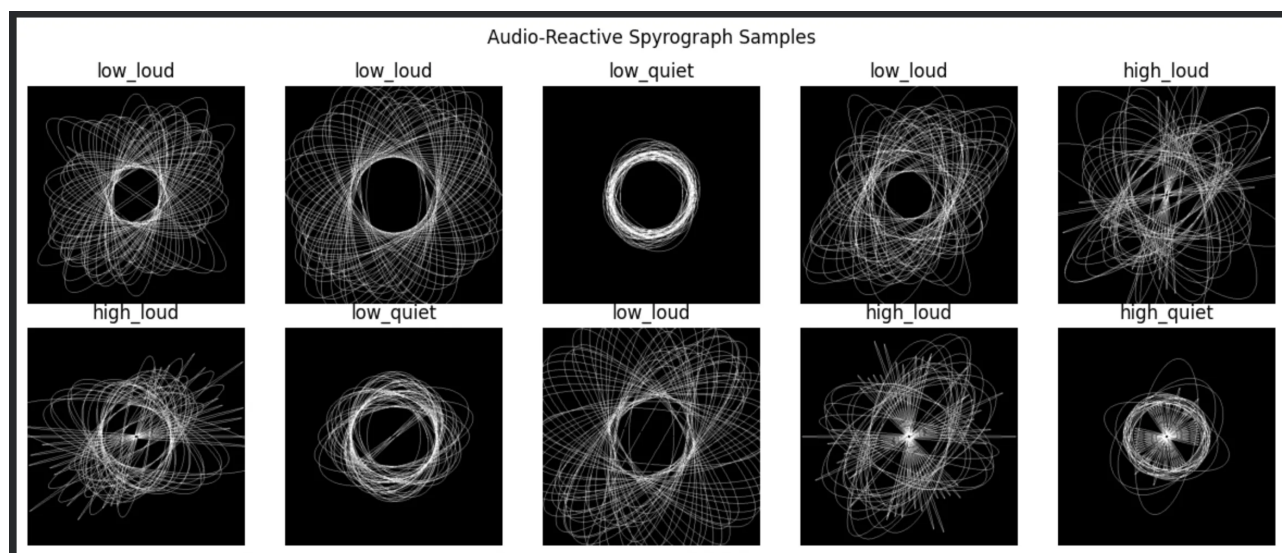
ELLA RAY



Computer generated audio-reactive drawings classified by a convolutional neural network into 4 classes: high_loud, high_quiet, low_loud, and low_quiet, representing pitch and amplitude. Then translated back into sound. This project explores a closed loop machine-machine communication from sound to drawing back to sound. A computer's interpretation of human expressive outputs i.e. drawing and music. What interested me most was the ambiguity that the machine had to understand.

Each spirograph represents a 2 second burst of sound, turned into a drawing by segmenting it into 50 frames and collecting the mean pitch and amplitude per frame. A frame was reflected by the spikiness and length of an oval. The spikier, the more high pitched and the bigger, the louder. This was to represent the collective human way of classifying sharper shapes as 'Kiki' and rounder shapes as 'Bouba', an experiment that showed the underlying classification system all humans share between abstract forms.

I chose to use a custom CNN, similar to a Le-Net model, for image classification because it needed to learn abstract geometric forms and it's good at extracting edges and shapes, plus it's good for small datasets.



The ambiguity of my dataset was my biggest issue resolved by creating bigger visual differences. I found that recording tracks instead of human audio inputs created better and more accurate visualisations. I toggled with exaggerating the spikiness and size of each oval so that the model could distinguish the classes. Initially, low-pitched but loud drawings became visually stretched and spiky, which caused them to resemble high-pitched drawings. This resulted in constant misclassification between high_loud and low_loud classes. It showed how the dataset design directly affected the model's performance.

Because pitch and amplitude are continuous, and classes are binary, the drawings that existed near the boundary were causing instability in validation accuracy and prediction jitter. The model in the end learnt that a wider gap in the middle meant low pitched and less empty space around the sides meant higher amplitude. It was a test of abstraction learning; visual cues that statistically correlated with these concepts rather than the concepts themselves.

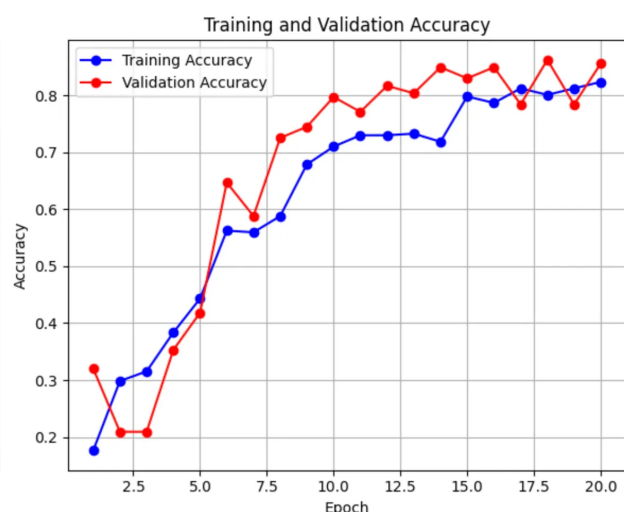
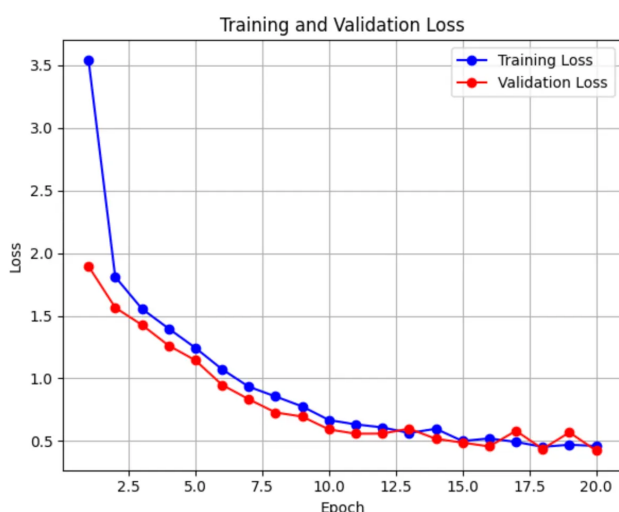
I transformed my dataset into greyscale images, downsampled from 600x600 to 256x256 to preserve shape and reduce computational load. Tested with various sizes but found it became too distorted when lower. I changed the interpolation mode to 'cubic', providing higher-quality, sharper and smoother images, as by default transform uses bilinear which made resizing more blurry.

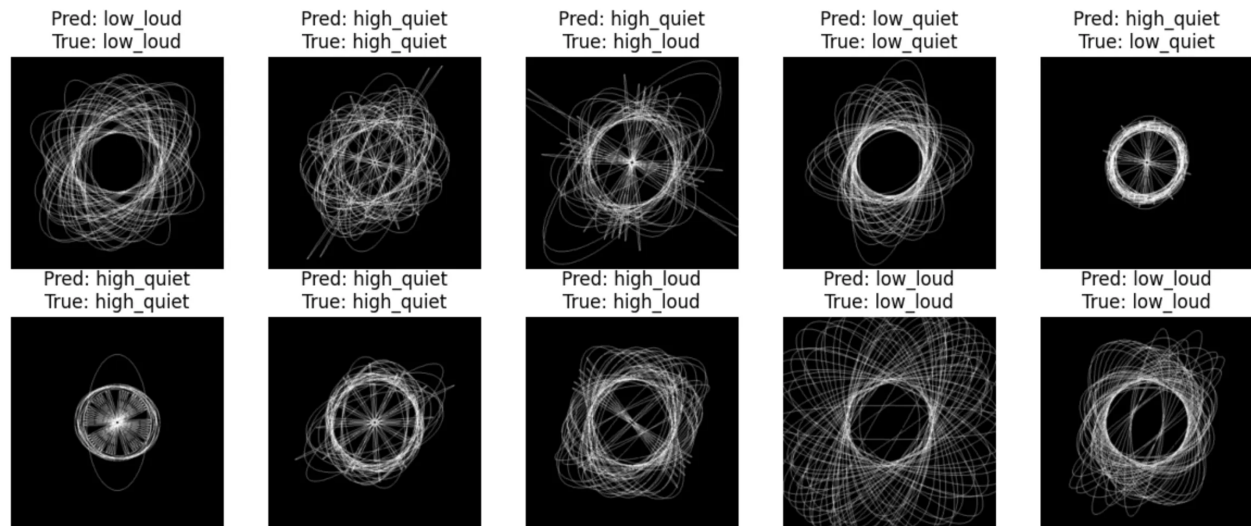
I started with 120 images, around 30 per class and increased it to around 350 images overall with a separate training dataset of 120 images due to the inaccuracy in the way it was learning. This improved generalisation hugely as there were less anomalies and drawings near the parameter boundary for it to learn with. I also redid my testing data as half of the drawings were from a previous audio-reactive drawing model.

I tested the training hyperparameters. I reduced the learning rate to 0.0005 from 0.001 to learn slower and more carefully which improved the accuracy from 68% to 73%, though this was on the old testing dataset. I incrementally increased the batch size from 8 to 32, the small batch size made the model jump around while increasing the batch size reduced the noise gradients for more stable updates, improving generalisation.

With 15 epochs, 0.0005 lr and 32 batch size I reached 84.31% which I thought was really good for inherently ambiguous data. With a batch size of 64 it reduced to 64% because it's a small dataset, though I tested it with 30 epochs to see if it needed more time to learn and it went up to 82.35%. I then tested epochs with a batch size of 32, it was overfitting with 30 epochs so the model learnt better with 20 epochs at 86%. I consistently used random rotation and horizontal flipping augmentations but didn't add more as I thought it was already abstract enough and would destruct the data.

I ended up using batch size of 32, 20 epochs and a 0.0005 learning rate, achieving a validation accuracy of 85.62%. The graphs below show the CrossEntropy loss over training and validation, showing that the model is learning the underlying patterns not just memorising the training dataset. Though there is some divergence between the training and validation accuracy due to the small noisy dataset, causing slight overfitting.



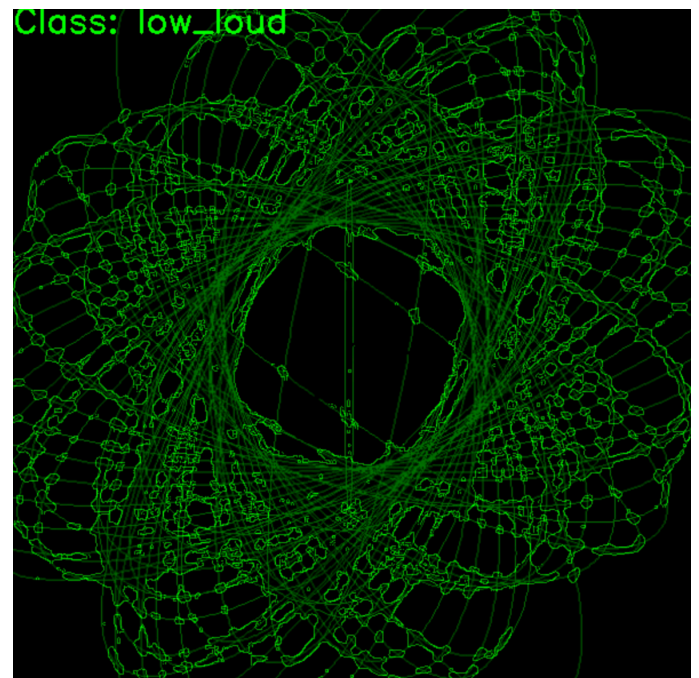


I downloaded this model and uploaded it onto my drawing_to_sound python file where I used Pytorch CNN for image classification, OpenCV for video capture and display and Pygame for real-time sound playback. My programme randomly selected a computer-generated sound from a corresponding class folder to match the spirograph visual as a sort of communication coming from the computer in a language it supposedly understands.

The computer-generated noises and green on black visual was to represent old digital models. I used OpenCV background subtraction and canny edge detection, with a median blur to reduce noise inbetween, for visual aesthetic as I really liked it in my element 1, assignment 1. I love the effect on the spirographs and think that the OpenCV techniques added a more interesting visual aesthetic that added to this older digital design, especially when intensified. People are more prone to interacting with older nostalgic models due to the embodied interfaces.

The final output of my code is an mp4 video animation of these spirograph drawings which my model classifies and simultaneously plays a correlating sound, experimenting with the objective perception of a machine and the ambiguity of abstract data.

The future for this project is a drawing machine resembling a record player that will listen and draw these spirographs physically with an x, y plotter similar to an etch a sketch. The 'record' it draws on will then be removable and placed onto another 'record player' that will utilise a camera instead of a tonearm. The model will 'read' the music from the drawing and output sounds whilst it spins. Adding a trigger line in my code for when the individual ovals are detected and pass it. This is a translation system between humans and machine about the objective classifications we all put on sound and shape.



After enhancing backSub + medianBlur + Canny